

uuuuu

Sebuah Mahakarya oleh uuu

# BAIT 1

```
'use server';

import { PDFDocument as PDFLib, StandardFonts, rgb, degrees } from 'pdf-lib';
import type { PDFFont, PDFImage, PDFPage } from 'pdf-lib';
import { promises as fs } from 'fs';
import path from 'path';
import { initializeFirebase } from '@/firebase';
import { collection, query, orderBy, getDocs, doc, getDoc, where } from 'firebase/firestore';
import type { Book, Chapter, User } from '@/lib/types';

const PAGE_WIDTH = 595.28;
const PAGE_HEIGHT = 841.89;
const MARGIN = 72;

// --- THEME ASSETS ---
const POEM_PAPER_TEXTURE_URL =
'https://media.istockphoto.com/id/654922866/photo/old-yellowed-stained-paper-texture.jpg?s=612x612&w=
const WATERMARK_URL =
'https://raw.githubusercontent.com/Zombiesigma/nusakarsa-v2/main/public/logo/copyright.png';

// --- THEME COLORS ---
const nusakarsaPrimary = rgb(0.45, 0.56, 0.22);
const nusakarsaBackground = rgb(0.97, 0.98, 0.96);
const defaultTextDark = rgb(0.1, 0.1, 0.1);
const textMuted = rgb(0.4, 0.4, 0.4);
const poemTextDark = rgb(0.24, 0.15, 0.14); // #3e2723

interface PageTheme {
  textColor: ReturnType<typeof rgb>;
  paperTexture?: PDFImage;
}

/
* Adds a background texture or color to a page.
*/
function addPageBackground(page: PDFPage, texture?: PDFImage, fallbackColor?:
  ReturnType<typeof rgb>) {
  const { width, height } = page.getSize();
  if (texture) {
    page.drawImage(texture, { x: 0, y: 0, width, height });
```

```

    } else if (fallbackColor) {
    page.drawRectangle({ x: 0, y: 0, width, height, color: fallbackColor });
    }
    }

    /
    * Adds a watermark to a PDF page.
    */
    function addWatermarkToPage(page:PDFPage, watermarkImage: PDFImage) {
    const { width, height } = page.getSize();
    const watermarkDims = watermarkImage.scale(0.8);

    page.drawImage(watermarkImage, {
    x: width / 2 - watermarkDims.width / 2,
    y: height / 2 - watermarkDims.height / 2,
    width: watermarkDims.width,
    height: watermarkDims.height,
    opacity: 0.05,
    rotate: degrees(-45),
    });
    }

    function sanitizePath(str: string): string {
    return str.replace(/^[a-z0-9]/gi, '_').toLowerCase().trim();
    }

    async function drawMarkdownParagraph(
    page: PDFPage,
    paragraph: string,
    y: number,
    doc: PDFLib,
    watermarkImage: PDFImage,
    theme: PageTheme
    ): Promise<{ y: number, page: PDFPage }> {
    let currentPage = page;
    let currentY = y;
    const contentWidth = PAGE_WIDTH - MARGIN * 2;

    const fontRegular = await doc.embedFont(StandardFonts.TimesRoman);
    const fontBold = await doc.embedFont(StandardFonts.TimesRomanBold);
    const fontItalic = await doc.embedFont(StandardFonts.TimesRomanItalic);

    let text = paragraph;
    let baseFont = fontRegular;

```

```

let baseSize = 12;
let xOffset = 0;

if (text.startsWith('# ')) { baseSize = 20; baseFont = fontBold; text = text.substring(2); }
else if (text.startsWith('## ')) { baseSize = 16; baseFont = fontBold; text = text.substring(3); }
else if (text.startsWith('### ')) { baseSize = 14; baseFont = fontBold; text = text.substring(4); }
}
else if (text.startsWith('> ')) { baseFont = fontItalic; text = text.substring(2); xOffset = 20; }

const lineHeight = baseSize * 1.4;
const tokens = text.split(/(\ \. ?\ \|\ \. ?\ )/g).filter(Boolean);
let currentX = MARGIN + xOffset;

for (const token of tokens) {
let font = baseFont;
let content = token;

if (token.startsWith(' ') && token.endsWith(' ')) { font = fontBold; content = token.slice(2, -2); }
else if (token.startsWith(' ') && token.endsWith(' ')) { font = fontItalic; content = token.slice(1, -1); }

const words = content.replace(/\n/g, ' \n ').split(' ');
for (const word of words) {
if (word === '\n') {
currentY -= lineHeight;
currentX = MARGIN + xOffset;
if (currentY < MARGIN) {
currentPage = doc.addPage([PAGE_WIDTH, PAGE_HEIGHT]);
addPageBackground(currentPage, theme.paperTexture);
addWatermarkToPage(currentPage, watermarkImage);
addFooter(currentPage, doc.getPageCount(), fontRegular, PAGE_WIDTH);
currentY = PAGE_HEIGHT - MARGIN;
}
continue;
}

if (!word) continue;

const wordWidth = font.widthOfTextAtSize(word, baseSize);

if (currentX + wordWidth > PAGE_WIDTH - MARGIN && currentX > MARGIN +
xOffset) {
currentY -= lineHeight;

```

```

currentX = MARGIN + xOffset;
}

if (currentY < MARGIN) {
  currentPage = doc.addPage([PAGE_WIDTH, PAGE_HEIGHT]);
  addPageBackground(currentPage, theme.paperTexture);
  addWatermarkToPage(currentPage, watermarkImage);
  addFooter(currentPage, doc.getPageCount(), fontRegular, PAGE_WIDTH);
  currentY = PAGE_HEIGHT - MARGIN;
  currentX = MARGIN + xOffset;
}

currentPage.drawText(word + ' ', {
  x: currentX, y: currentY, font: baseSize, color: theme.textColor,
});
currentX += font.widthOfTextAtSize(word + ' ', baseSize);
}
}

currentY -= lineHeight;
return { y: currentY, page: currentPage };
}

export async function generateBookPdf(bookId: string): Promise<string> {
  const { firestore } = initializeFirebase();
  if (!firestore) throw new Error('Firestore not initialized');

  const bookSnap = await getDoc(doc(firestore, 'books', bookId));
  if (!bookSnap.exists()) throw new Error('Book not found');
  const book = { id: bookSnap.id, ...bookSnap.data() } as Book;

  const chaptersQuery = query(collection(firestore, 'books', bookId, 'chapters'), orderBy('order', 'asc'));
  const chaptersSnap = await getDocs(chaptersQuery);
  const chapters = chaptersSnap.docs.map(d => ({ id: d.id, ...d.data() } as Chapter));

  const pdfDoc = await PDFLib.create();

  // --- ASSET FETCHING ---
  const watermarkBytes = await fetch(WATERMARK_URL).then(res => res.arrayBuffer());
  const watermarkImage = await pdfDoc.embedPng(watermarkBytes);

  let coverImage: PDFImage | undefined;
  if (book.coverUrl) {
    try {

```

```

const coverBytes = await fetch(book.coverUrl).then(res => res.arrayBuffer());
coverImage = await pdfDoc.embedJpg(coverBytes);
} catch (e) { console.warn("Failed to embed cover image", e); }
}

const isPoem = book.type === 'poem';
let pageTheme: PageTheme = { textColor: defaultTextDark };
if (isPoem) {
  try {
    const paperBytes = await fetch(POEM_PAPER_TEXTURE_URL).then(res =>
    res.arrayBuffer());
    pageTheme.paperTexture = await pdfDoc.embedJpg(paperBytes);
    pageTheme.textColor = poemTextDark;
  } catch(e) { console.warn("Failed to embed poem paper texture.", e); }
}

const fontHeadline = await pdfDoc.embedFont(StandardFonts.TimesRomanBold);
const fontSerifRegular = await pdfDoc.embedFont(StandardFonts.TimesRoman);

// --- CINEMATIC COVER PAGE ---
let coverPage = pdfDoc.addPage([PAGE_WIDTH, PAGE_HEIGHT]);
const { width, height } = coverPage.getSize();

if (coverImage) {
  const imgDims = coverImage.scaleToCover(width, height);
  coverPage.drawImage(coverImage, { x: 0, y: 0, width, height, opacity: 0.8 });
  coverPage.drawRectangle({ x: 0, y: 0, width, height, color: rgb(0,0,0), opacity: 0.4 });
} else {
  coverPage.drawRectangle({ x: 0, y: 0, width, height, color: nusakarsaBackground });
  addWatermarkToPage(coverPage, watermarkImage);
}

const titleLines = wrapText(book.title, width - (MARGIN * 2), fontHeadline, 42);
let currentY = height - 220;
for (const line of titleLines) {
  coverPage.drawText(line, {
    x: MARGIN, y: currentY, font: fontHeadline, size: 42, color: rgb(1,1,1),
    maxWidth: width - MARGIN * 2, lineHeight: 50,
    wordBreaks: [ ' ' ]
  });
  currentY -= 50;
}

coverPage.drawLine({ start: { x: MARGIN, y: currentY }, end: { x: width - MARGIN, y:

```

```

currentY }, thickness: 0.5, color: rgb(1,1,1), opacity: 0.5 });
currentY -= 30;

coverPage.drawText(`Sebuah Mahakarya oleh ${book.authorName}`, {
x: MARGIN, y: currentY, font: fontSerifRegular, size: 16, color: rgb(0.9, 0.9, 0.9)
});

const footerText = `Diterbitkan melalui Nusakarsa © ${new Date().getFullYear()}`;
coverPage.drawText(footerText, {
x: MARGIN, y: MARGIN, size: 9, font: fontSerifRegular, color: rgb(0.8, 0.8, 0.8)
});

// --- CONTENT PAGES ---
let contentPage = pdfDoc.addPage([PAGE_WIDTH, PAGE_HEIGHT]);
addPageBackground(contentPage, pageTheme.paperTexture);
addWatermarkToPage(contentPage, watermarkImage);
addFooter(contentPage, pdfDoc.getPageCount(), fontSerifRegular, width);

let contentY = height - MARGIN;
const fontSerifBold = await pdfDoc.embedFont(StandardFonts.TimesRomanBold);

for (const chapter of chapters) {
const spaceNeeded = 22 / title / + 20 / margin / + 24 / first line estimate /;
if (contentY < MARGIN + spaceNeeded) {
contentPage = pdfDoc.addPage([PAGE_WIDTH, PAGE_HEIGHT]);
addPageBackground(contentPage, pageTheme.paperTexture);
addWatermarkToPage(contentPage, watermarkImage);
addFooter(contentPage, pdfDoc.getPageCount(), fontSerifRegular, width);
contentY = height - MARGIN;
}

const chapterTitleLines = wrapText(chapter.title, width - MARGIN*2, fontSerifBold, 22);
for(const line of chapterTitleLines) {
contentPage.drawText(line, {
x: MARGIN, y: contentY, size: 22, font: fontSerifBold, color: pageTheme.textColor
});
contentY -= 28;
}

contentY -= 30;

const paragraphs = chapter.content.replace(/\r\n/g, '\n').split(/\n\s*\n/);
for (const para of paragraphs) {
if (!para.trim()) continue;

```

```

const { y: newY, page: newPage } = await drawMarkdownParagraph(contentPage, para,
contentY, pdfDoc, watermarkImage, pageTheme);
contentY = newY;
contentPage = newPage;
contentY -= 12; // Space between paragraphs

if (contentY < MARGIN) {
contentPage = pdfDoc.addPage([PAGE_WIDTH, PAGE_HEIGHT]);
addPageBackground(contentPage, pageTheme.paperTexture);
addWatermarkToPage(contentPage, watermarkImage);
addFooter(contentPage, pdfDoc.getPageCount(), fontSerifRegular, width);
contentY = height - MARGIN;
}
}
contentY -= 20; // Extra space after chapter
}

const pdfBytes = await pdfDoc.save();
const pdfBuffer = Buffer.from(pdfBytes);
const safeFileName = `${sanitizePath(book.title)}.pdf`;
const typeFolder = isPoem ? 'puisi' : 'buku';
const folderPath = `${typeFolder}/${sanitizePath(book.title)}`;

return await uploadPdf(pdfBuffer, safeFileName, folderPath);
}

async function uploadPdf(buffer: Buffer, fileName: string, folderPath: string):
Promise<string> {
const GITHUB_TOKEN = process.env.GITHUB_TOKEN;
const GITHUB_REPO_OWNER = process.env.GITHUB_REPO_OWNER;
const GITHUB_REPO_NAME = process.env.GITHUB_REPO_NAME;

if (GITHUB_TOKEN && GITHUB_REPO_OWNER && GITHUB_REPO_NAME) {
try {
return await uploadPdfToGithub(buffer, fileName, folderPath, GITHUB_TOKEN,
GITHUB_REPO_OWNER, GITHUB_REPO_NAME);
} catch (e) {
console.warn("[PDF Generator] GitHub upload failed, falling back...");
}
}

try {
return await uploadToPublicService(buffer, fileName);
} catch (e) {

```

```

console.error("[PDF Generator] All upload methods failed:", e);
throw new Error("Gagal mengunggah file PDF.");
}
}

```

```

async function uploadPdfToGithub(buffer: Buffer, fileName: string, folderPath: string, token:
string, owner: string, repo: string): Promise<string> {
const base64Content = buffer.toString('base64');
const timestamp = Date.now();
const filePath = `${folderPath}/${timestamp}-${fileName}`;

```

```

const response = await
fetch(`https://api.github.com/repos/${owner}/${repo}/contents/${filePath}`, {
method: 'PUT',
headers: {
'Authorization': `Bearer ${token}`,
'Content-Type': 'application/json',
'Accept': 'application/vnd.github.v3+json',
'User-Agent': 'Nusakarsa-App',
},
body: JSON.stringify({
message: `Automatic PDF Generation for ${fileName}`,
content: base64Content,
branch: 'main'
}),
signal: AbortSignal.timeout(110000),
});

```

```

if (!response.ok) {
const errorData = await response.json().catch(() => ({}));
throw new Error(errorData.message || 'GitHub Error');
}

```

```

return `https://raw.githubusercontent.com/${owner}/${repo}/main/${filePath}`;
}

```

```

async function uploadToPublicService(buffer: Buffer, fileName: string): Promise<string> {
const POMF_MIRRORS = ['https://pomf.lain.la/upload.php', 'https://quax.moe/upload.php',
'https://pomf.cat/upload.php'];
for (const mirror of POMF_MIRRORS) {
try {
const formData = new FormData();
const blob = new Blob([buffer], { type: 'application/pdf' });

```

```

formData.append('files[]', blob, fileName);
const response = await fetch(mirror, { method: 'POST', body: formData, signal:
AbortSignal.timeout(30000) });
if (response.ok) {
const data = await response.json();
if (data.success && data.files && data.files[0]) return data.files[0].url;
}
} catch (err) { continue; }
}
throw new Error("Public storage mirrors failed.");
}

function addFooter(page: any, pageNum: number, font: any, width: number) {
page.drawText(`${pageNum}.`, { x: width - MARGIN, y: 40, size: 9, font: font, color:
textMuted });
}

function wrapText(text: string, maxWidth: number, font: any, fontSize: number): string[] {
if (!text) return [""];
const paragraphs = text.split('\n');
const allLines: string[] = [];
for (const para of paragraphs) {
if (!para.trim()) { allLines.push(""); continue; }
const words = para.split(/\s+/);
let currentLine = "";
for (const word of words) {
const testLine = currentLine ? `${currentLine} ${word}` : word;
const testWidth = font.widthOfTextAtSize(testLine, fontSize);
if (testWidth > maxWidth) {
allLines.push(currentLine);
currentLine = word;
} else { currentLine = testLine; }
}
allLines.push(currentLine);
}
return allLines;
}

```